

Damage conversion discards the source element's increases

Element-gated damage multipliers apply only to the unconverted share of a hit, so converting damage throws away every increase you had for the element you converted *from*. If conversion is meant to follow Path of Exile semantics, this is inverted.

MOD	PACK	SEVERITY
Mine & Slash 1.20.1-6.4.13	Craft to Exile 2 (2.0 Atlas)	Build-breaking
FOUND VIA		
In-game + bytecode		

What happens

A character built around physical damage takes a conversion talent (for example the Rogue physical→chaos passive, or the Shaman spider talent). Damage does not stay level and it does not improve — it drops sharply, and at full conversion the skill does almost nothing.

The cause is that increases are gated on the element of the hit *after* conversion. Every element-scoped multiplier — `all_physical_damage`, `all_chaos_damage`, and the rest — carries `ifs: ["ele_match_stat"]`, and that condition reads:

```
// ElementMatchesStat.can(...)
if (damageEvent.unconvertedDamagePercent <= 0) return false;
if (damageEvent.unconvertedDamageTakenAsPercent <= 0) return false;
return event.getElement().elementsMatch(stat.getElement());
```

The event tracks what fraction of the hit is still unconverted, and an element-matched multiplier applies only to that fraction. Convert everything and `unconvertedDamagePercent` is zero, so the source element's multipliers return `false` and contribute nothing at all.

Worked example

A hit with 100 base physical damage, on a character with **+100% increased physical damage** and **no chaos scaling**. Expected column assumes PoE-style conversion, where increases that applied before conversion continue to apply after it.

CONVERTED TO CHAOS	EXPECTED	ACTUAL	SHORTFALL
0%	200	200	—
50%	200	150	−25%
100%	200	100	−50%

At 50% conversion the surviving physical half still gets the multiplier ($50 \times 2 = 100$) while the converted half gets nothing ($50 \times 1 = 50$). At 100% the multiplier is switched off entirely and the hit lands for its raw base.

THE EXPECTATION

If 100 physical becomes 200 after a +100% increase, converting that damage to chaos should deal 200 chaos — the conversion moves the damage, not the scaling that has already been applied to it. Today it deals 100.

Why it can't currently work the other way

This looks structural rather than a single bad comparison. `DamageEvent` carries only two relevant fields:

```
float unconvertedDamagePercent;  
float unconvertedDamageTakenAsPercent;  
int conversionDepth;           // guards chains, < 2
```

There is no record of *which* element a converted share came from — only how much of the hit is unconverted, as a single scalar. So the engine has no way to answer “is this chaos damage that used to be physical?”, which is the question PoE-style conversion needs to ask. Supporting it would mean tracking conversion provenance per element rather than one aggregate percentage.

Reproduction

- 1 Build a character with significant increased physical damage and no chaos scaling.
- 2 Note the tooltip or dummy DPS for a physical skill (Magic Missile reproduces it).
- 3 Allocate a physical→chaos conversion source, such as the Rogue conversion passive.
- 4 Re-check damage. It falls roughly in proportion to the percentage converted.
- 5 Push total conversion to 100% or beyond. Damage drops to approximately the unscaled base.

The training dummy makes this measurable without a target: the drop is visible on a single skill with no other build changes.

Two related observations

OVER-100% CONVERSION IS SILENTLY DISCARDED

`Conversion.normalizeNumbersToCapTo100()` scales every conversion source by `100 / total` so the shares sum to exactly 100. That is reasonable, but nothing in-game indicates it — a player stacking a third conversion source sees no warning that the investment does nothing, and no indication that they can no longer retain any of the original element.

TWO STATS READ ALMOST IDENTICALLY AND BEHAVE OPPOSITELY

`plus_phys_to_<ele>` (`BonusPhysicalAsElemental`, “Grants % of physical attack damage as extra elemental damage”) is additive and leaves physical untouched. `phys_to_<ele>` (`PhysicalToElement`, “Turns % of phys atk dmg into ele”) is real conversion. The Venom runeword grants the first; the Rogue passive grants the second. The in-game wording does distinguish them, but the similarity is easy to misread — clearer separation in tooltips would help.

Proposed fix

One detail makes this cheaper to fix than it looks: `all_physical_damage` and `all_chaos_damage` are typed `MULTIPLICATIVE_DAMAGE` but their effect is `_additive_damage_number_add_stat_data` — sources pool additively and the pool applies once. That is the same shape as a Path of Exile “increased” modifier, which means the cheap fix below is exact rather than an approximation.

Tier 1 — track provenance, widen the gate

Three changes, no restructuring:

1. Add lineage to `DamageEvent`. It currently holds one scalar, `unconvertedDamagePercent`. Add `Map<Elements, Float> convertedFrom` — how much of the hit originated as each element.
2. Populate it in `StatLayerData.convertDamage()`, which already receives the destination element and the percentage. The source element is the event’s element at that moment, so the information is present and simply discarded today.
3. Widen `ElementMatchesStat.can()` from “matches the current element” to “is anywhere in this hit’s lineage”.

```
// was: return event.getElement().elementsMatch(stat.getElement());
return event.getElement().elementsMatch(stat.getElement())
    || damageEvent.convertedFrom.containsKey(stat.getElement());
```

Because increases pool additively before being applied, this gives exactly the expected result whenever a hit converts to a single destination — 100 physical with +100% increased stays 200 at 0%, 50% or 100% conversion. That covers effectively every real build.

Where it stays imprecise: a hit converting to two different destinations with different increases. The whole hit would receive every applicable increase rather than each share receiving its own. Over-generous, but far closer than the current behaviour.

Tier 2 — split the hit into buckets

The correct general model. Instead of one event with one element and one number, produce one bucket per final element, each carrying its share of the base and its lineage set. Run the multiplier stack per bucket — an element-gated stat applies if its element is in *that bucket’s* lineage — then sum the buckets.

Larger, but it fixes three things beyond this report:

- Brutality (`all_chaos_damage -100% MORE`) zeroes only the chaos share, rather than interacting unpredictably with conversion.
- Penetration and resistance resolve per final element.
- The damage breakdown becomes explainable to players, because each share has a traceable stack.

Two adjacent notes

- `conversionDepth < 2` prevents conversion chains. If the intent is PoE semantics, conversion runs Physical → Lightning → Cold → Fire → Chaos in that fixed order and damage can pass through several steps, accumulating increases at each. That guard blocks it.
- `normalizeNumbersToCapTo100()` already matches PoE, which also scales conversion down proportionally past 100%. No change needed there.

THIS IS A BALANCE EVENT, NOT ONLY A FIX

Every existing conversion build becomes meaningfully stronger the moment this ships, and conversion changes from a trap into a genuinely competitive option. Worth weighing before release rather than after.

How this was determined. Observed in play on a level 100 Rogue/Hunter character, then traced through decompiled bytecode of `Mine_and_Slash-1.20.1-6.4.13.jar`: `ElementMatchesStat`, `PhysicalToElement`, `BonusPhysicalAsElemental`, `StatLayerData$Conversion` and `DamageEvent`.

Bytecode reading can misrepresent intent, and the “expected” column assumes conversion is meant to follow Path of Exile semantics. If that assumption is wrong and the current behaviour is deliberate, the substantive request narrows to making it legible in-game: conversion away from your scaled element is currently an invisible, respec-priced trap.